INSTITUTO TECNOLÓGICO SUPERIOR DEL ORIENTE DEL ESTADO DE HIDALGO

Ingeniería en Sistemas Computacionales.

Docente: Efrén Rolando Romero León

Asignatura: Métodos numéricos

Evidencia: Errores de programación

Grupo:2

Alumnos:

Briana Alexa Del Valle Guzmán 24030220

Alan Aguilar De Lucio 24030053

Cristhian Bernardo Garcia Mora 24030063

Periodo: Enero – Julio

Contenido

Desarrollo....3
1.- Error de Redondeo Binario....3
2. Pérdida de Precisión por Magnitud(IEEE 754)....5
3. Comparación Directa con ==....7
4. Acumulación de Errores en Bucles....9
5. Cancelación por Resta (Loss of Significance)....11
7. Conversión Estrecha (Narrowing Primitive Conversion)....15
8. Excel/ software....17
Evidencia de trabajo en equipo:....18
Diagrama de Gantt....18
Conclusiones....19
Referencias....20

Desarrollo

1.- Error de Redondeo Binario

Ejemplo 1: Suma de decimales

Código:

```
public class Main {

    public static void main(String[] args) {

        double resultado = 0.1 + 0.2;

        System.out.println(resultado);

    }

}
```

Salida:

```
0.30000000000000004
```

Explicación: 0.1 y 0.2 no tienen representación binaria exacta, causando error de redondeo.

Ejemplo 2: Multiplicación decimal

Código:

```
public class Main {

    public static void main(String[] args) {

        double resultado = 0.1 * 3;

        System.out.println(resultado);

    }

}
```

Salida:

```
0.30000000000000004
```

Explicación: Multiplicar 0.1 tres veces acumula el error de representación binaria.

Ejemplo 3: Resta de decimales

Código:

```java
public class Main {

    public static void main(String[] args) {

        double resultado = 1.0 - 0.9;

        System.out.println(resultado);

    }

}
```

Salida:

```
0.09999999999999998
```

Explicación: El sistema binario no puede representar exactamente 0.9, generando imprecisión.

2. Pérdida de Precisión por Magnitud(IEEE 754)

Ejemplo 1: Número grande + pequeño

Código:

```
public class Main {
    public static void main(String[] args) {
        double grande = 1.0e16;
        double pequeno = 1.0;
        double resultado = grande + pequeno;
        System.out.println(resultado == grande);
    }
}
```

Salida: true

Explicación: El double solo tiene 15-17 dígitos significativos. El +1 se pierde completamente.

Ejemplo 2: Suma repetida con gran magnitud

Código:

```
public class Main {
    public static void main(String[] args) {
        double suma = 1.0e15;
        for(int i = 0; i < 100; i++) {
        suma += 0.1;
```

```
        }
        System.out.println(suma);
    }
}
```

Salida: `1.00000000000000125E15`

Explicación: 100 sumas de 0.1 desaparecen porque la mantisa no tiene bits suficientes.

Ejemplo 3: Diferencia de magnitudes

Código:

```
public class Main {
    public static void main(String[] args) {
        double grande = 9999999999999999.0;
        double pequeno = 0.5;
        System.out.println((grande + pequeno) - grande);
    }
}
```

Salida: `0.0`

Explicación: Al sumar y luego restar el número grande, el pequeño se pierde por falta de precisión.

3. Comparación Directa con ==

Ejemplo 1: Comparación directa fallida

Código:

```
public class Main {
    public static void main(String[] args) {
        double a = 0.1 + 0.1 + 0.1;
        double b = 0.3;
        System.out.println(a == b);
    }
}
```

Salida: `false`

Explicación: Los errores de redondeo hacen que a y b sean diferentes. Usar:

Math.abs(a - b) < 0.00001

Ejemplo 2: División y comparación

Código:

```
public class Main {
    public static void main(String[] args) {
        double a = 1.0 / 10.0;
        double b = 0.1;
        System.out.println(a == b);
    }
}
```

Salida: true

Explicación: Aunque funciona aquí, no es confiable. Diferentes operaciones pueden generar diferentes errores.

Ejemplo 3: Solución con épsilon

Código:

```
public class Main {
    public static void main(String[] args) {
        double a = 0.1 + 0.1 + 0.1;
        double b = 0.3;
        double epsilon = 0.00001;
        System.out.println(Math.abs(a - b) < epsilon);
    }
}
```

Salida: true

Explicación: Usar un margen de error (épsilon) es la forma correcta de comparar números decimales.

4. Acumulación de Errores en Bucles

Ejemplo 1: Suma en bucle

Código:

```
public class Main {

    public static void main(String[] args) {

        double suma = 0.0;

        for(int i = 0; i < 10; i++){

        suma += 0.1;

        }

        System.out.println(suma);

    }

}
```

Salida:

```
0.9999999999999999
```

Explicación: Cada suma de 0.1 tiene un pequeño error que se acumula en el bucle.

Ejemplo 2: Cálculo de interés

Código:

```
public class Main {

    public static void main(String[] args) {

        double saldo = 100.0;

        for(int i = 0; i < 1000; i++){

        saldo += 0.01;

        }

        System.out.println(saldo);
```

```
    }

}
```

Salida: `110.00000000000512`

Explicación: Después de 1000 sumas, el error acumulado es visible. En finanzas esto es crítico.

Ejemplo 3: Solución con BigDecimal

Código:

```
public class Main {

    public static void main(String[] args) {

        BigDecimal suma = BigDecimal.ZERO;

        BigDecimal inc = new BigDecimal("0.1");

        for(int i = 0; i < 10; i++) {

        suma = suma.add(inc);

        }

        System.out.println(suma);

    }

}
```

Salida: `1.0`

Explicación: BigDecimal mantiene precisión exacta y evita errores de acumulación en bucles.

5. Cancelación por Resta (Loss of Significance)

Código:

```
public class Main {
  public static void main(String[] args) {
    // Dos números muy cercanos entre sí
    double a = 1.000000;
    double b = 0.9999999999;
    // Operación propensa a la cancelación
    double resultado = a - b;
    System.out.println("Valor de a: " + a);
    System.out.println("Valor de b: " + b);
    System.out.println("Resultado de (a - b): " + resultado);
    // El resultado matemático exacto debería ser 0.0000000001 (1.0E-10)
  }
}
```

Salida:

```
Valor de a: 1.0
Valor de b: 0.9999999999
Resultado de (a - b): 1.000000082740371E-10
```

Valor de a: 1.0

Valor de b: 0.9999999999

Resultado de (a - b): 1.000000082740371E-10

Explicación Representación Binaria: Los números como 0.9999999999 no pueden representarse de forma exacta en el sistema binario que usa Java (IEEE 754). Se guardan como una aproximación muy cercana.

Cómo solucionarlo

- BigDecimal: Para cálculos financieros o donde la precisión absoluta sea crítica, usa la clase BigDecimal de Java.

6. Desbordamiento Silencioso (Overflow)

Ejemplo 1: Suma que desborda

```
public class Main {

    public static void main(String[] args) {

        int a = 2_000_000_000;  // 2 mil millones

        int b = 500_000_000;    // 500 millones

        int suma = a + b;

        System.out.println("a = " + a);

        System.out.println("b = " + b);

        System.out.println("Suma = " + suma);

    }

}
```

Salida:

```
a = 2000000000
b = 500000000
Suma = -1794967296
```

Explicación:

La suma debería ser 2,500,000,000 (2.5 mil millones)

int solo puede guardar hasta 2,147,483,647

Al exceder este límite, "da la vuelta" y se vuelve negativo

Es como un odómetro que llega a 999,999 y vuelve a 000,000

Ejemplo 2: Desbordamiento en multiplicación

```java
public class Main {

    public static void main(String[] args) {

        int segundosPorDia = 86_400;     // 24 horas * 60 min * 60 seg

        int dias = 30;                    // 30 días


        int segundosTotales = segundosPorDia * dias;


        System.out.println("Segundos por día: " + segundosPorDia);

        System.out.println("Días: " + dias);

        System.out.println("Total segundos: " + segundosTotales);

    }

}
```

Solución:

```
Segundos por día: 86400
Días: 30
Total segundos: 2592000
```

Explicación:

86,400 × 365 × 100 = 3,153,600,000

Esto excede Integer.MAX_VALUE (2,147,483,647)

El resultado es completamente incorrecto por overflow

7. Conversión Estrecha (Narrowing Primitive Conversion)

Ejemplo 1: Conversión de double a int (perdiendo decimales)

```
public class Main {
    public static void main(String[] args) {
double precio = 99.99;
        int precioEntero = (int) precio;

        System.out.println("Precio original: $" + precio);
        System.out.println("Precio convertido: $" + precioEntero);
        System.out.println("Dinero perdido: $" + (precio -
precioEntero));
    }
}
```

Solución:

```
Precio original: $99.99
Precio convertido: $99
Dinero perdido: $0.9899999999999949
```

Explicación:
Al convertir double a int, Java **CORTA** los decimales (NO redondea)
Se pierden los 99 centavos
(int) 99.99 = 99, NO 100
Esto es peligroso en cálculos financieros

Ejemplo 2: Conversión de double grande a int (truncamiento al límite)

```
public class Main {
    public static void main(String[] args) {
        double distanciaKm = 5_000_000_000.0;  // 5 mil millones de km
        int distanciaInt = (int) distanciaKm;

        System.out.println("Distancia real: " + distanciaKm + " km");
        System.out.println("Distancia en int: " + distanciaInt + "
km");

        // Verificar si se truncó
        if (distanciaInt == Integer.MAX_VALUE) {
            System.out.println("¡ADVERTENCIA! El valor fue truncado al
máximo de int");
        }
    }
}
```

Otro lenguaje:

Ruby

```ruby
distancia_km = 5_000_000_000.0  # 5 mil millones de km
distancia_int = distancia_km.to_i

puts "Distancia real: #{distancia_km} km"
puts "Distancia en int: #{distancia_int} km"

# Verificar si se truncó
# Nota: En Ruby, los enteros NO tienen límite fijo como en Java
# Ruby usa "Bignum" automáticamente para números grandes
if distancia_int == distancia_km.to_i
  puts "En Ruby, ¡los enteros pueden ser tan grandes como quieras!"
  puts "No hay truncamiento porque Ruby maneja números grandes
automáticamente"
end
```

Solución:

```
Distancia real: 5.0E9 km
Distancia en int: 2147483647 km
¡ADVERTENCIA! El valor fue truncado al máximo de int
```

Explicación:

5,000,000,000 es mucho más grande que Integer.MAX_VALUE (2,147,483,647)
Java NO puede meter ese número en un int
En lugar de dar error, simplemente lo **trunca al máximo** que puede
Pierdes completamente el valor real

8. Excel/ software

Ejemplo 2: Cancelación en Excel

En la celda A1, escribe: 1.000000

En la celda B1, escribe: 0.9999999999

En la celda C1, pon la fórmula: =A1-B1

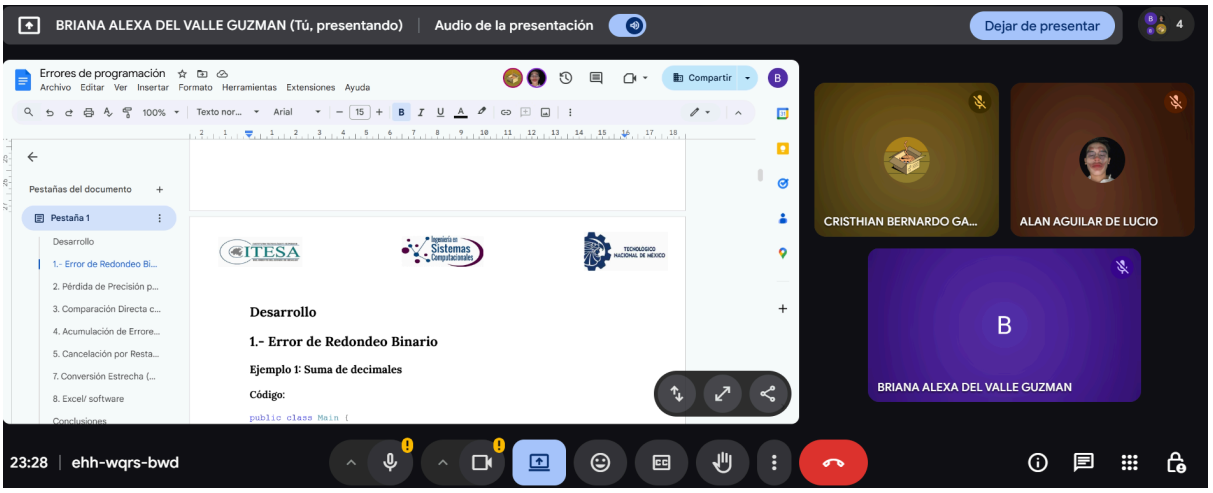
A simple vista, Excel podría mostrarte 0.0000000001. Pero aquí está el truco para ver el error:

Selecciona la celda C1 > pestaña Inicio > Número y aumenta la cantidad de decimales (haz clic varias veces en el icono de la flecha azul hacia la izquierda .00 -> .000).

Cuando llegues a unos 15 o 17 decimales, verás algo como esto:

0.000000000100000000002875...

Evidencia de trabajo en equipo:



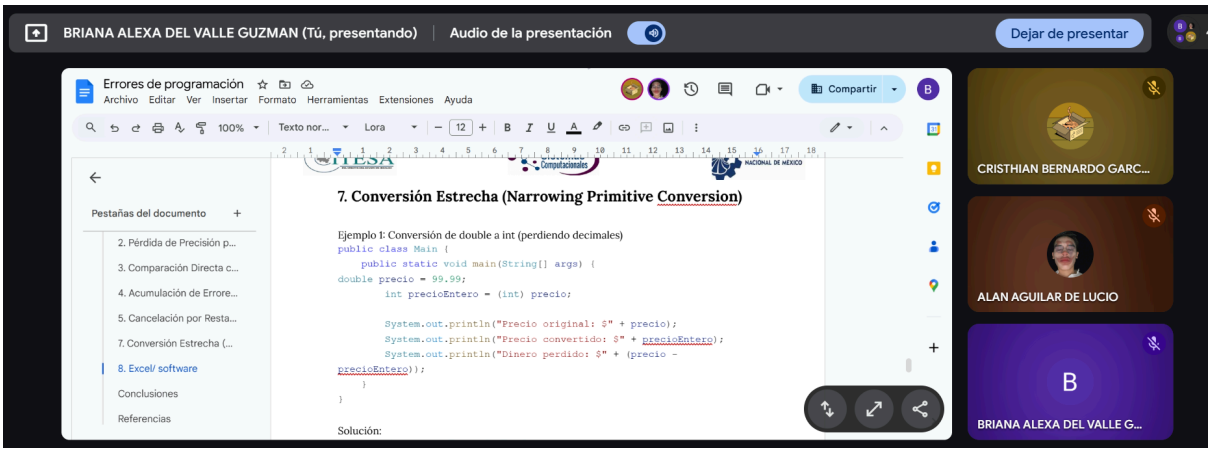


Diagrama de Gantt

	ENERO		
TAREAS	Briana	Alan	Cristhian
1.- Error de Redondeo Binario			
2. Pérdida de Precisión por Magnitud(IEEE 754)			
3. Comparación Directa con ==			
4. Acumulación de Errores en Bucles			
5. Cancelación por Resta (Loss of Significance)			
6. Desbordamiento Silencioso (Overflow)			
7. Conversión Estrecha (Narrowing Primitive Conversion)			
8. Excel/ software			

Conclusiones

Los errores numéricos en programación son inevitables debido a la forma en que las computadoras representan los números en sistema binario. Identificar estos errores es fundamental para desarrollar software confiable, especialmente en áreas críticas como finanzas, ciencia e ingeniería, donde la precisión es esencial. Comprender el redondeo binario, la pérdida por magnitud, los problemas de comparación directa y la acumulación de errores en bucles nos permite tomar decisiones informadas al elegir tipos de datos apropiados (como BigDecimal para operaciones financieras) y aplicar técnicas adecuadas (como comparaciones con épsilon). Este conocimiento nos ayuda a prevenir errores costosos, garantizar resultados exactos en cálculos críticos y escribir código más robusto y profesional.

Referencias

Qué es un error de desbordamiento? Cómo evitarlo en programación | Lenovo España. (2023, 28 mayo). https://www.lenovo.com/es/es/glossary/overflow-error/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsltid=AfmBOop1zSjvydEWEnxOlqmk9sAxMK_Xo0mMNzIqYCtHr3seIVck2bMP

Casero, A. (2024, 15 marzo). Entendiendo los errores de conversión en Python. *KeepCoding Bootcamps*. https://keepcoding.io/blog/errores-de-conversion-en-python/